RLP Circuit (RLP 인코딩 검증 회로)

개요

RLP (Recursive Length Prefix) Circuit은 이더리움 트랜잭션의 RLP 인코딩을 검증하는 회로입니다. 이 회로는 유한 상태 머신(FSM, Finite State Machine) 기반으로 구현되어 있으며, RLP 인코딩 규칙에 따라 트랜잭션 데이터를 파싱하고 검증합니다.

주요 역할:

- RLP 인코딩된 트랜잭션 바이트 검증
- 트랜잭션 필드별 데이터 추출 및 검증
- 중첩된 리스트 구조 처리 (Access List 등)
- 가스 비용 계산 및 RLC (Random Linear Combination) 생성

파일 위치: `zkevm-circuits/src/rlp_circuit_fsm.rs`

RLP 인코딩 규칙

RLP는 이더리움에서 데이터를 직렬화하는 표준 방식입니다. 바이트 값의 범위에 따라 다른 인코딩 규칙이 적용됩니다:

기본 인코딩 규칙

1. **단일 바이트 (0x00 ~ 0x7f)**: 값 자체가 인코딩
2. **짧은 바이트열 (0x80 ~ 0xb7)**:

- `0x80 + length` 형식
- 예: 길이 5인 바이트열 → `0x85` + 데이터

3. **긴 바이트열 (0xb8 ~ 0xbf)**:

- `0xb7 + length_of_length` + length + 데이터
- 예: 길이 1024인 바이트열 → `0xb9 + 0x0400` + 데이터

4. **짧은 리스트 (0xc0 ~ 0xf7)**:

- `0xc0 + total_length` 형식
- 예: 총 길이 10인 리스트 → `0xca` + 리스트 항목들

5. **긴 리스트 (0xf8 ~ 0xff)**:

- `0xf7 + length_of_length` + total_length + 리스트 항목들

특수 케이스

- **빈 바이트열**: `0x80`
- **빈 리스트**: `0xc0`
- **NULL 값**: `0x80` 으로 인코딩

FSM 기반 상태 머신

RLP Circuit은 다음 상태들로 구성된 유한 상태 머신으로 동작합니다:

상태 정의

```
pub enum State {
DecodeTagStart, // 새로운 태그 디코딩 시작
Bytes, // 짧은 바이트열 처리
LongBytes, // 긴 바이트열의 길이 읽기
LongList, // 긴 리스트의 길이 읽기
```

```
End, // 디코딩 완료
}
```

상태 전이 다이어그램

```
DecodeTagStart
├─> Bytes (0x80 < byte < 0xb8)
│  └─> DecodeTagStart (태그 완료)
├─> LongBytes (0xb8 <= byte < 0xc0)
│  ├─> LongBytes (길이 읽기 중)
│  └─> Bytes (길이 읽기 완료)
├─> DecodeTagStart (0xc0 <= byte < 0xf8, 짧은 리스트)
├─> LongList (0xf8 <= byte)
│  ├─> LongList (길이 읽기 중)
│  └─> DecodeTagStart (길이 읽기 완료, depth 증가)
└─> End (depth == 0 && is_tag_end)
```

주요 컴포넌트

1. RLP 테이블 (RlpFsmRlpTable)

다른 회로에서 접근 가능한 공개 테이블입니다:

```rust
pub struct RlpFsmRlpTable {
```

```
pub q_enable: Column<Fixed>,
pub tx_id: Column<Advice>,
pub format: Column<Advice>,
pub rlp_tag: Column<Advice>,
pub tag_value: Column<Advice>,
pub tag_bytes_rlc: Column<Advice>,
pub tag_length: Column<Advice>,
pub is_output: Column<Advice>,
pub is_none: Column<Advice>,
pub access_list_idx: Column<Advice>,
pub storage_key_idx: Column<Advice>,
}
```

필드 설명:

- `tx_id` : 트랜잭션 ID
- `format` : 트랜잭션 포맷 (PreEip155, Eip155, Eip1559 등)
- `rlp_tag` : 현재 필드 태그 (Nonce, GasPrice, To 등)
- `tag_value` : 필드의 값
- `tag_bytes_rlc` : 필드 바이트의 RLC
- `tag_length` : 필드 길이
- `is_output` : 출력 여부 (다른 회로에서 사용 가능)
- `is_none` : NULL 값 여부
- `access_list_idx` : Access List 인덱스

- `storage_key_idx` : Storage Key 인덱스

2. 데이터 테이블 (RlpFsmDataTable)

RLP 인코딩된 원본 바이트 데이터를 저장합니다:

```
pub struct RlpFsmDataTable {
pub tx_id: Column<Advice>,
pub format: Column<Advice>,
pub byte_idx: Column<Advice>,
pub byte_rev_idx: Column<Advice>,
pub byte_value: Column<Advice>,
pub bytes_rlc: Column<Advice>,
pub gas_cost_acc: Column<Advice>,
}
```

필드 설명:

- `byte_idx` : 바이트 인덱스 (1부터 시작)
- `byte_rev_idx` : 역방향 인덱스 (끝에서부터)
- `byte_value` : 바이트 값
- `bytes_rlc` : 누적 RLC
- `gas_cost_acc` : 누적 가스 비용

3. ROM 테이블 (RlpFsmRomTable)

트랜잭션 포맷별 필드 정의를 저장하는 읽기 전용 테이블입니다:

```rust
pub struct RlpFsmRomTable {
pub tag: Column<Fixed>,
pub tag_next: Column<Fixed>,
pub max_length: Column<Fixed>,
pub is_list: Column<Fixed>,
pub format: Column<Fixed>,
}
```

필드 설명:

- `tag` : 현재 필드 태그
- `tag_next` : 다음 필드 태그
- `max_length` : 필드의 최대 길이
- `is_list` : 리스트 여부
- `format` : 트랜잭션 포맷

4. 디코딩 테이블 (RlpDecodingTable)

중첩된 리스트 구조를 처리하기 위한 스택 제약 조건을 관리합니다:

```rust
pub struct RlpDecodingTable {
pub id: Column<Advice>,
pub tx_id: Column<Advice>,
pub format: Column<Advice>,
pub depth: Column<Advice>,
pub byte_idx: Column<Advice>,
```

```
pub value: Column<Advice>,
pub value_prev: Column<Advice>,
pub is_stack_init: Column<Advice>,
pub is_stack_push: Column<Advice>,
pub is_stack_pop: Column<Advice>,
pub is_stack_update: Column<Advice>,
pub al_idx: Column<Advice>,
pub sk_idx: Column<Advice>,
}
```

스택 연산:

- `Init` : 스택 초기화 (depth 0에서 시작)
- `Push` : 새로운 리스트 시작 (depth 증가)
- `Pop` : 리스트 종료 (depth 감소)
- `Update` : 현재 depth에서 바이트 읽기

FSM 동작 원리

DecodeTagStart 상태

새로운 필드를 디코딩하기 시작하는 상태입니다. 바이트 값에 따라 다음 상태로 전이합니다:

Case 1: 단일 바이트 (0x00 ~ 0x7f)

```
- 값 자체가 필드 값
- 태그 출력 (is_output = true)
```

- 다음 태그로 전이

Case 2: 빈 바이트열 (0x80)

- NULL 값 (is_none = true)
- 태그 출력
- 다음 태그로 전이

Case 3: 짧은 바이트열 (0x80 < byte < 0xb8)

- tag_length = byte_value - 0x80
- Bytes 상태로 전이
- 다음 바이트부터 데이터 읽기 시작

Case 4: 긴 바이트열 (0xb8 <= byte < 0xc0)

- tag_length = byte_value - 0xb7 (길이의 길이)
- LongBytes 상태로 전이
- 실제 길이를 읽기 위해 추가 바이트 필요

Case 5: 짧은 리스트 (0xc0 <= byte < 0xf8)

- 리스트 길이 = byte_value - 0xc0
- depth 증가
- DecodeTagStart로 재귀 (리스트 내부 처리)

Case 6: 긴 리스트 (0xf8 <= byte)

```
- tag_length = byte_value - 0xf7 (길이의 길이)
- LongList 상태로 전이
- 실제 길이를 읽기 위해 추가 바이트 필요
```

Bytes 상태

짧은 바이트열의 데이터를 읽는 상태입니다:

```
// tag_idx < tag_length인 동안 반복
tag_value_acc = tag_value_acc * base + byte_value_next
tag_bytes_rlc = tag_bytes_rlc * r + byte_value_next
tag_idx += 1

// tag_idx == tag_length일 때
- 태그 출력 (tag_value = tag_value_acc)
- DecodeTagStart로 전이
```

base 값 결정:

- `max_length < 32` : base = 256 (일반 정수)
- `max_length == 32` : base = evm_word_rand (EVM word)
- `max_length > 32` : base = keccak_input_rand (해시 입력)

LongBytes 상태

긴 바이트열의 길이를 읽는 상태입니다:

```
// tag_idx < tag_length인 동안 반복
tag_value_acc = tag_value_acc * 256 + byte_value_next
tag_idx += 1

// tag_idx == tag_length일 때
- 실제 데이터 길이 = tag_value_acc
- Bytes 상태로 전이
- 데이터 읽기 시작
```

LongList 상태

긴 리스트의 길이를 읽는 상태입니다:

```
// tag_idx < tag_length인 동안 반복
tag_value_acc = tag_value_acc * 256 + byte_value_next
tag_idx += 1

// tag_idx == tag_length일 때
- 리스트 총 길이 = tag_value_acc
- depth 증가
- DecodeTagStart로 전이
- 리스트 내부 항목 처리 시작
```

중첩 구조 처리

Depth 관리

RLP Circuit은 중첩된 리스트를 처리하기 위해 depth를 사용합니다:

- **depth = 0**: 최상위 레벨 (트랜잭션 전체)
- **depth = 1**: 트랜잭션 필드 레벨
- **depth = 2**: Access List 레벨
- **depth = 3**: Access List 항목 레벨
- **depth = 4**: Storage Key 리스트 레벨

스택 제약 조건

디코딩 테이블은 스택과 유사한 구조로 `remaining_bytes` 를 추적합니다:

PUSH 연산 (리스트 시작):

```
- 새로운 리스트의 remaining_bytes를 스택에 푸시
- depth 증가
- 리스트 내부 항목 처리 시작
```

POP 연산 (리스트 종료):

```
- remaining_bytes가 0이 되면 리스트 종료
- 스택에서 팝
- depth 감소
```

UPDATE 연산 (바이트 읽기):

```
- 바이트를 읽을 때마다 remaining_bytes 감소
- 같은 depth 유지
```

Access List 처리

Access List는 EIP-2930에서 도입된 기능으로, 트랜잭션이 접근할 주소와 스토리지 키를 미리 선언합니다:

```
AccessList = [
[address1, [storage_key1, storage_key2, ...]],
[address2, [storage_key3, storage_key4, ...]],
...
]
```

인덱스 관리:

- `access_list_idx` : 현재 처리 중인 주소 인덱스
- `storage_key_idx` : 현재 처리 중인 스토리지 키 인덱스

인덱스 증가 조건:

- `is_new_access_list_address` : 새로운 주소 시작 시 `access_list_idx++`
- `is_new_access_list_storage_key` : 새로운 스토리지 키 시작 시 `storage_key_idx++`

인덱스 초기화 조건:

- `is_access_list_end` (depth == 2 && EndVector): `access_list_idx = 0`
- `is_storage_key_list_end` (depth == 4 && EndVector): `storage_key_idx = 0`

제약 조건

1. 데이터 테이블 제약 조건

초기화 제약:

```
byte_idx[0] = 1

bytes_rlc[0] = byte_value[0]
```

연속성 제약 (같은 트랜잭션 내):

```
byte_idx' = byte_idx + 1

byte_rev_idx' + 1 = byte_rev_idx

bytes_rlc' = bytes_rlc * r + byte_value'
```

트랜잭션 경계 제약:

```
// 마지막 바이트

byte_rev_idx = 1


// 새로운 트랜잭션 시작

byte_idx' = 1

bytes_rlc' = byte_value'
```

2. 상태 전이 제약 조건

각 상태에서 다음 상태로의 전이는 엄격한 제약 조건을 따릅니다:

DecodeTagStart → Bytes:

```
조건: 0x80 < byte_value < 0xb8

제약:

- is_list = false

- tag_length = byte_value - 0x80

- tag_idx' = 1

- state' = Bytes
```

Bytes → DecodeTagStart:

```
조건: tag_idx == tag_length

제약:

- tag_length <= max_length

- is_output = true

- tag_value = tag_value_acc

- state' = DecodeTagStart
```

DecodeTagStart → LongList:

```
조건: byte_value >= 0xf8

제약:

- is_tag_begin = true

- tag_length = byte_value - 0xf7

- tag_idx' = 1
```

```
- state' = LongList
```

LongList → DecodeTagStart:

```
조건: tag_idx == tag_length

제약:

- tag_length <= max_length

- depth' = depth + 1

- state' = DecodeTagStart

- (depth == 0일 때) is_output = true, rlp_tag = Len
```

3. ROM 테이블 룩업

모든 (tag, tag_next, max_length, is_list, format) 조합은 ROM 테이블에 존재해야 합니다:

```
meta.lookup_any("ROM table lookup", |meta| {

vec![

tag,

tag_next,

max_length,

is_list,

format,

]

.into_iter()

.zip(rom_table.table_exprs(meta))
```

```
.map(|(arg, table)| (cond * arg, table))
.collect()
});
```

4. 스택 제약 조건

PUSH-POP 대응:

```
// 모든 PUSH는 대응하는 POP이 있어야 함
lookup: (byte_idx + value, tx_id, format, depth - 1, is_pop = true, value + 1)
→ (byte_idx, tx_id, format, depth, is_stack_pop, value_prev - value)

// 모든 POP은 대응하는 PUSH가 있어야 함
lookup: (byte_idx - (value_prev - value - 1), tx_id, format, depth + 1, is_push
→ (byte_idx, tx_id, format, depth, is_stack_push, value)
```

UPDATE 제약:

```
value + 1 = value_prev // 1바이트 읽음
value[prev] = value_prev[cur] // 바이트를 건너뛰지 않음
```

5. 바이트 값 범위 제약

바이트 값에 따른 상태 전이를 위해 여러 비교 제약 조건이 사용됩니다:

```
byte_value <= 0x80
```

```
byte_value >= 0x80

byte_value <= 0xb8

byte_value >= 0xb8

byte_value <= 0xc0

byte_value >= 0xc0

byte_value <= 0xf8

byte_value >= 0xf8
```

이러한 비교는 `ComparatorChip` 을 사용하여 구현됩니다.

출력 및 룩업

RLP 테이블 출력

`is_output = true` 인 행만 다른 회로에서 사용할 수 있습니다:

출력되는 태그:

- 트랜잭션 필드 값 (Nonce, GasPrice, To, Value 등)
- `RlpTag::Len` : 트랜잭션 전체 길이
- `RlpTag::RLC` : 트랜잭션 바이트의 RLC
- `RlpTag::GasCost` : 트랜잭션 가스 비용

Tx Circuit과의 연동

Tx Circuit은 RLP 테이블을 룩업하여 트랜잭션 필드를 검증합니다:

```
// Tx Circuit에서 RLP 테이블 룩업
```

```
lookup: (tx_id, format, rlp_tag, tag_value, ...)

→ RlpFsmRlpTable
```

최적화 기법

1. 차수 감소 (Degree Reduction)

복잡한 조건을 미리 계산하여 회로 차수를 줄입니다:

```
// 차수 감소를 위한 불리언 컬럼

is_tag_end = is_tag_end_object || is_tag_end_vector

is_tag_begin = is_tag_begin_object || is_tag_begin_vector

is_case3 = (0xc0 <= byte_value < 0xf8) && !is_tag_end

transit_to_new_rlp_instance = is_tag_end && (depth == 0) && (state' != End)

is_same_rlp_instance = (tx_id' == tx_id) && (format' == format)
```

2. 이진 분해 (Binary Decomposition)

상태와 태그를 이진수로 분해하여 효율적인 비교를 가능하게 합니다:

```
state_bits: BinaryNumberConfig<State, 3> // 3비트로 상태 표현

tag_bits: BinaryNumberConfig<Tag, 5> // 5비트로 태그 표현
```

3. RLC 계산

Random Linear Combination을 사용하여 바이트 시퀀스를 단일 필드 원소로 압축합니다:

```
// 데이터 테이블에서
bytes_rlc' = bytes_rlc * keccak_input_rand + byte_value'

// 태그 바이트에서
tag_bytes_rlc' = tag_bytes_rlc * keccak_input_rand + byte_value'

// 태그 값에서 (max_length에 따라 다른 challenge 사용)
tag_value_acc' = tag_value_acc * base + byte_value'
```

트랜잭션 포맷 지원

RLP Circuit은 다양한 트랜잭션 포맷을 지원합니다:

1. PreEip155 (Legacy)

```
RLP([nonce, gasPrice, gas, to, value, data, v, r, s])
```

2. Eip155

```
RLP([nonce, gasPrice, gas, to, value, data, v, r, s])
// v 값에 chain_id 포함
```

3. Eip2930 (Type 1)

```
0x01 || RLP([chainId, nonce, gasPrice, gas, to, value, data, accessList, v, r, s
```

4. Eip1559 (Type 2)

```
0x02 || RLP([chainId, nonce, maxPriorityFeePerGas, maxFeePerGas, gas, to, value,
```

5. L1Msg (Scroll 특화)

```
// L1에서 L2로 전송되는 메시지
```

각 포맷은 ROM 테이블에 정의된 필드 순서와 제약 조건을 따릅니다.

예시: 간단한 트랜잭션 디코딩

입력 데이터

```
RLP 인코딩: 0xc8 0x01 0x02 0x03 0x80 0x04 0xc2 0x05 0x06
```

디코딩 과정

Step 1: `byte_idx=1, byte_value=0xc8`

```
상태: DecodeTagStart

조건: 0xc0 <= 0xc8 < 0xf8 (짧은 리스트)

동작:
```

```
- 리스트 길이 = 0xc8 - 0xc0 = 8

- depth: 0 → 1

- 출력: RlpTag::Len, tag_value = 9 (전체 길이)

- 다음 상태: DecodeTagStart
```

Step 2: `byte_idx=2, byte_value=0x01`

```
상태: DecodeTagStart

조건: 0x00 <= 0x01 < 0x80 (단일 바이트)

동작:

- tag_value = 0x01

- 출력: 현재 태그 (예: Nonce)

- 다음 상태: DecodeTagStart
```

Step 3: `byte_idx=3, byte_value=0x02`

```
상태: DecodeTagStart

조건: 0x00 <= 0x02 < 0x80 (단일 바이트)

동작:

- tag_value = 0x02

- 출력: 다음 태그 (예: GasPrice)

- 다음 상태: DecodeTagStart
```

Step 4: `byte_idx=4, byte_value=0x03`

```
상태: DecodeTagStart

조건: 0x00 <= 0x03 < 0x80 (단일 바이트)

동작:

- tag_value = 0x03

- 출력: 다음 태그 (예: Gas)

- 다음 상태: DecodeTagStart
```

Step 5: `byte_idx=5, byte_value=0x80`

```
상태: DecodeTagStart

조건: byte_value == 0x80 (빈 바이트열)

동작:

- tag_value = 0, is_none = true

- 출력: 다음 태그 (예: To, NULL)

- 다음 상태: DecodeTagStart
```

Step 6: `byte_idx=6, byte_value=0x04`

```
상태: DecodeTagStart

조건: 0x00 <= 0x04 < 0x80 (단일 바이트)

동작:

- tag_value = 0x04

- 출력: 다음 태그 (예: Value)
```

```
- 다음 상태: DecodeTagStart
```

Step 7: `byte_idx=7, byte_value=0xc2`

```
상태: DecodeTagStart

조건: 0xc0 <= 0xc2 < 0xf8 (짧은 리스트)

동작:

- 리스트 길이 = 0xc2 - 0xc0 = 2

- depth: 1 → 2

- 다음 상태: DecodeTagStart
```

Step 8-9: `byte_idx=8-9, byte_value=0x05, 0x06`

```
상태: DecodeTagStart (각각)

조건: 단일 바이트

동작:

- 리스트 내부 항목 처리

- depth = 2 유지
```

Step 10: 리스트 종료

```
상태: DecodeTagStart

조건: is_tag_end && depth == 2

동작:

- depth: 2 → 1
```

```
- 다음 상태: DecodeTagStart
```

관련 문서

- Tx Circuit: RLP 테이블을 사용하는 주요 회로
- Keccak Circuit: RLC 계산에 사용되는 challenge 제공
- 테이블 시스템: 회로 간 데이터 공유 메커니즘

참고 자료

- RLP 인코딩 사양
- EIP-2930: Optional Access Lists
- EIP-1559: Fee Market
- RLP Circuit 설계 문서